SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

Minor Project

Course Code: 1010103494

AI-Powered Digital Fraud Risk Analyzer

Project ID	M2_CE_054
Enrollment No	2402030430005, 2402030430007
Name	Jagatiya Anjna, Gupta Soni
Branch	Computer Engineering (CE)
Guide Name	Prof. Khushbu Modi

Index

1 Introduction

2 Background and Motivation

3 Relevance and Importance

4 Literature Review

5 Objectives

6 Required Tools and Technology

7 Method / Approach

8 Bibliography

Introduction

The rapid growth of online banking, digital payments, and e-commerce has been matched by a rise in digital fraud — phishing websites, scam messages, malicious QR codes, and fake reward links designed to steal money or personal data.

Most users only discover a scam after losing money — there is no easy way to check a suspicious link, message, or QR code before acting on it.

Websites / URLs

Phishing links, brand-impersonation domains

Messages

SMS, email, and chat-based scam text

QR Codes

Payment-redirection and fake merchant codes

This project proposes an AI-Powered Digital Fraud Risk Analyzer — a web platform that analyzes suspicious content submitted by a user and returns a plain-language fraud risk score before harm occurs.

Background and Motivation

- Digital payment adoption in India has grown sharply through UPI, net banking, and e-commerce, expanding the surface area for fraud.
- Common attacks — phishing URLs, lottery/reward SMS scams, and malicious payment QR codes — rely on urgency and impersonation rather than technical exploits, making them hard for an average user to spot.
- Existing protection is fragmented: browsers flag some known-bad URLs, but there is no single, approachable tool that lets a non-technical user check a link, message, or QR code and get a clear explanation of why it looks risky.
- This gap motivates a tool that combines AI-based pattern detection with explainable, actionable output — so users understand the 'why' behind a risk score, not just a pass/fail verdict.

Relevance and Importance

New insight this project contributes: a single, explainable risk-scoring layer that works across three common fraud vectors (URL, message, QR) instead of a narrow, single-purpose detector.

Everyday digital users

Anyone receiving unfamiliar links, offers, or payment QR codes

First-time online-banking users

Especially those newly onboarded to UPI and net banking, with less exposure to scam patterns

Small businesses & individuals

Verifying a payment QR or link before accepting or forwarding it

Why it's worth doing: fraud losses are frequently preventable at the decision point — the moment before a click, reply, or payment. Giving users a fast, understandable risk check at exactly that moment has outsized impact relative to after-the-fact remediation.

Literature Review

URL / phishing detection	Feature-based classifiers (domain age, lexical structure, SSL metadata) and blocklist approaches are widely used, but static blocklists lag behind newly registered scam domains.
Message / SMS scam detection	NLP-based classifiers trained on urgency cues, reward language, and link patterns perform well for text, but are rarely combined with URL or QR analysis in one tool.
QR code fraud	A comparatively newer research area; most existing work focuses on malware-laced QR payloads rather than payment-redirection scams common in UPI ecosystems.
Explainable AI in security tools	SHAP and similar attribution methods are increasingly used to make model decisions interpretable, addressing the 'black box' trust gap common in fraud-detection systems.

Objectives

1. Design and build a unified web platform that accepts a URL, message, or QR code image as input for fraud risk analysis.

2. Develop an AI/ML scoring pipeline (XGBoost) that outputs a 0–100 fraud risk score across all three input types.

3. Integrate SHAP-based explainability so every score is accompanied by clear, human-readable reasons.

4. Provide actionable recommendations calibrated to risk level, avoiding false certainty ('guaranteed safe/unsafe').

5. Maintain a scan history and report view so users can revisit and track past analyses.

Required Tools and Technology

Frontend

- Next.js + TypeScript
- Tailwind CSS
- shadcn/ui, Lucide icons

Backend Services

- Spring Boot (Gateway, Auth, Report services)
- JWT-based authentication
- REST APIs

AI / ML Service

- Python, FastAPI
- XGBoost for risk scoring
- SHAP for explainability
- SMOTE for class-imbalance handling

Data & Infra

- Database for scan history / reports
- Git & GitHub for version control
- Postman for API testing

Method / Approach

A microservice architecture separates concerns between presentation, orchestration, and AI inference — allowing the ML model to be improved independently of the frontend or auth layer.

Frontend

User submits URL, message, or QR image

→

Gateway Service

Spring Boot — routes and orchestrates requests

→

AI Service

Python — feature extraction + XGBoost scoring + SHAP

→

Report Service

Stores result, returns score & explanation

Development phases

- Data collection & labeling of known phishing URLs, scam message samples, and QR payment patterns
- Feature engineering (domain structure, urgency language, destination mismatch) and model training with XGBoost
- SHAP-based explainability layer to surface top contributing factors per prediction
- Frontend integration via REST APIs, with mock services replaced by live endpoints incrementally

Bibliography

References will be indicated by number(s) in square brackets in line with the text, and listed in full below as the literature review is finalized.

[1] — Phishing URL detection using lexical and host-based features (to be finalized with specific source).

[2] — SMS/text-based scam classification using NLP techniques (to be finalized with specific source).

[3] — QR code security and payment-redirection fraud patterns (to be finalized with specific source).

[4] — Lundberg, S. & Lee, S. — A Unified Approach to Interpreting Model Predictions (SHAP).

[5] — Chen, T. & Guestrin, C. — XGBoost: A Scalable Tree Boosting System.

Complete System Architecture

End-to-end design: user input, processing services, storage, and returned output.

